

Project: Food Remedy API

Ticket: RE-030 – Android Mobile Application Final Testing and Evidence Guidelines

Author: Gurkirat Singh Chahal

Student ID: 223936693

Date: 27/09/2026

Table of Contents

1. Introduction
2. Final Android Testing Areas
3. Network, Offline and Failure Testing
4. Physical Device vs Emulator Testing
5. Stability and Responsiveness Testing
6. Testing Evidence and Defect Recording
7. Recommendations
8. Conclusion
9. References

1. Introduction

Before an Android application is deemed ready for release, final testing is a crucial step. Its goal is to verify that the key features of the application work as intended, that it is stable in both usual and unforeseen circumstances, and that major flaws are found before the release is made available to consumers.

The current Android quality requirements place a strong emphasis on assessing the responsiveness, compatibility, and stability of applications. Major application functionality should be tested on supported Android environments, and an application should perform without crashes or Application Not Responding (ANR) problems.

Final testing for Food Remedy should concentrate on the primary user journeys in addition to scenarios like device variations, invalid responses, application restarts, and inaccessible internet. In order for the team to ascertain what was tested, if it passed, and what flaws still exist, testing should also yield enough proof.

This study offers a straightforward checklist that the Food Remedy team can use to assess a release candidate and identifies useful final Android testing procedures.

2. Final Android Testing Areas

Functionality that directly impacts the user's ability to use the programme should be the focus of final testing.

Table 1. Important Final Testing Areas

Testing Area	Example Check
Installation	Release candidate installs successfully
App launch	Application opens without crashing
Navigation	Main screens and navigation operate correctly
Authentication	Login/logout and account functions work
Product search	Products can be searched and displayed
Barcode/product functions	Product lookup returns expected information
Food information	Ingredients and nutrition information display correctly
Allergen information	Available allergen information displays correctly
Favourites	Products can be added/removed correctly
Shopping lists	List operations function as expected
Network behaviour	App handles unavailable/slow connections
Restart	Important state remains correct after closing/reopening
Error handling	Failures produce understandable behaviour/messages
Performance	Main screens remain responsive
Stability	No unexpected crashes or ANRs

Google's Android quality guidelines advise going over the main functionalities of the application and making sure it doesn't crash or block the UI thread. Checking startup and overall responsiveness is also advised.

Therefore, rather of testing individual screens separately, Food Remedy's final testing should be focused mostly on comprehensive user journeys.

3. Network, Offline and Failure Testing

A mobile application cannot make the assumption that there will always be a dependable internet connection. Therefore, both normal connectivity and failure conditions should be included in final testing.

Table 2. Network and Failure Tests

Scenario	Expected Behaviour
----------	--------------------

Normal internet	Online features operate normally
Internet disconnected	App does not crash
Connection lost during operation	Failure is handled appropriately
Connection restored	User can continue/retry where appropriate
Slow connection	Loading state or appropriate feedback is shown
API request fails	User receives suitable error behaviour
Empty response	App handles missing results safely
App closed/reopened	Required state/data remains consistent
Repeated actions	Duplicate or unintended operations are avoided

Features that synchronise data or access product information should receive special consideration for Food Remedy.

Making every feature function entirely offline is not always the goal. Rather, when network-dependent functionality is not available, the programme should fail in a safe and predictable manner.

Closing and reopening the programme after significant operations should be part of the testing process. This can highlight issues with local storage, saved state, or synchronisation that might not show up during a continuous testing session.

4. Physical Device vs Emulator Testing

Although they have different uses, emulators and real Android smartphones are both beneficial.

Table 3. Physical Device vs Emulator

Area	Emulator	Physical Device
Android versions	Excellent	Limited to available devices
Screen sizes	Excellent	Limited
Quick repeat testing	Excellent	Good
Real hardware behaviour	Limited/simulated	Excellent
Camera/barcode testing	Limited/simulated	Excellent
Real network behaviour	Simulated	Excellent
Real performance	Approximation	More representative
Installation experience	Good	Realistic
Final release verification	Useful	Essential

Before releasing an application, Android documentation specifically advises testing it on a real device. For testing various Android versions and screen sizes, the Android Emulator is still helpful.

Consequently, Food Remedy ought to employ both.

More coverage across Android versions and device combinations can be offered by emulators.

For the final release candidate, physical devices should be utilised, especially for practical features like camera/barcode interaction, connectivity, and overall responsiveness.

Services like Firebase Test Lab may also test applications across several physical and virtual devices when more extensive device coverage is required.

5. Stability and Responsiveness Testing

Final testing should confirm that Food Remedy maintains stability as users carry out its primary tasks.

Crucial inspections consist of:

- There are no unforeseen programme crashes;
- No ANR mistakes
- The programme successfully launches;
- Screens react to user input;
- Appropriate feedback is provided by loading operations;
- Failures are not caused by repetitive navigation;
- The application functions properly when it is backgrounded and reopened; and
- The application is not frozen or terminated by common error circumstances.

Crashes and ANRs are recognised by Android as critical stability indicators. While pre-release testing should try to find these problems beforehand, Android Vital tracks user-perceived crash and ANR rates after deployment.

Additionally, concerns including crashes, ANRs, sluggish startup, memory problems, and device compatibility issues can be found in Google Play's pre-launch reports. Google points out that these automated checks cannot ensure that every problem will be found.

As a result, automated tests should complement the Food Remedy team's manual final testing rather than take its place.

6. Testing Evidence and Defect Recording

Testing should yield sufficient data so that a different team member can comprehend what was tested and what transpired.

Suitable Testing Evidence

Evidence	Example
Screenshot	Successful product result or error message

Screen recording	Complete user flow
Test checklist	Pass/fail result for each check
Device information	Device model and Android version
Build information	APK/version/build tested
Logs	Relevant crash/error output
Test date	When testing occurred
Tester	Person performing the test

Excessive evidence is not necessary. While brief recordings can be helpful when illustrating navigation or multi-step activity, screenshots are acceptable for visual outcomes.

When a flaw is found, the team should document enough details so that another person can replicate it.

An example of a defect record

Defect: After a failed lookup, the product page crashes

Build: First Release Candidate

Device: Samsung Galaxy S23

Version of Android: 16

Steps: Launch the scanner, scan the product, disconnect the internet, and try again.

Anticipated: An error message appeared

Real: The application is closed.

Level of severity: high

Proof: Screen capture plus pertinent log

Current state: Open

A fault should, at the very least, document:

Build, device, Android version, reproduction instructions, predicted and actual results, severity, and proof

Instead of only proving that testing took place, this makes the final testing proof valuable for development.

7. Recommendations

A straightforward, repeatable release-testing procedure should be used by Food Remedy:

Release Candidate → Functional Testing → Device Testing → Network/Failure Testing → Stability Check → Documentation → Fix Critical Defects → Retest → Release Decision

It is advised to follow these procedures:

1. Examine the real release candidate. Refrain from depending solely on outcomes from previous development builds.
2. Give critical user journeys top priority. Particular focus should be given to product lookup, food information, allergies, and important user-data operations.
3. Consciously test failure conditions. Restart the programme, disconnect the network, and attempt any unsuccessful operations.
4. Test with both hardware devices and emulators. While physical devices offer more realistic final verification, emulators offer configuration coverage.
5. Don't accept ANRs or crashes that don't make sense. Before the build is deemed ready, they should be looked into.
6. Consistently document testing. Retain the build, device, Android version, outcome, and pertinent evidence.
7. Retest the fixed flaws. After a flaw is fixed, the identical situation should be used for verification.
8. When possible, use Google Play's automatic testing. The team's manual testing on additional devices can be enhanced by pre-launch reports.

8. Conclusion

Before a release candidate is accepted, final Android testing should verify that Food Remedy's primary functionality functions well and stays stable under actual circumstances.

Important user journeys, network and offline behaviour, programme restart, error situations, crashes, ANRs, and responsiveness should all be tested. Physical device testing is a crucial component of final verification, and both emulators and physical devices should be employed.

Documentation of testing is also necessary. The team can show what was tested and replicate failures when needed thanks to screenshots, recordings, test results, build information, and defect logs.

A straightforward release checklist offers Food Remedy a useful method for ensuring final testing is consistent across releases. Only once crucial functionality has been confirmed, serious flaws have been fixed, and relevant testing data has been gathered should a release candidate move forward.

References

- **Android Developers. (2026). Core app quality guidelines. Google.**
<https://developer.android.com/docs/quality-guidelines/core-app-quality>
- Useful for functional, stability, performance, crash and ANR testing.
<https://developer.android.com/docs/quality-guidelines/core-app-quality>
- **Android Developers. (2026). Run apps on a hardware device. Google.**
<https://developer.android.com/studio/run/device>
- **Android Developers. (2026). Prepare your app for release. Google.**
<https://developer.android.com/studio/publish/preparing>